Quantum Computing, Quantum Machine Learning and Quantum Information Theories

Morten Hjorth-Jensen[1,2]

Department of Physics, University of Oslo[1]

Department of Physics and Astronomy and Facility for Rare Isotope Beams, Michigan State University[2]

March 26, 2025

Plans for the week of March 24-28, 2025

1. Start discussion of discrete Fourier transforms and Quantum Fourier transforms, basic mathematical expressions
2. Reading recommendation Hundt, Quantum Computing for Programmers, sections 6.1-6.4 on QFT.

Possible paths for project 2

1. Implement QFTs and study the quantum phase estimation (QPE) algorithm and
   - 1.1 Compare QFTs with Fast Fourier transforms
   - 1.2 Add a comparison of QPE for finding eigenvalues with the VQE method from project 1 or
   - 1.3 Study Shor's algorithm for factorization of numbers
2. Study other algorithms (you may need QFTs and QPE)
   - 2.1 Deutsch-Jozsa algorithm: Determine if a function is constant or balance using the fewest number of queries.
   - 2.2 Other algorithms
3. Study the solution of quantum mechanical eigenvalue problems with systems from atomic/molecular physics and quantum chemistry using adaptive QPE
4. Quantum machine learning projects like quantum Boltzmann machine or quantum neural networks
5. Other ideas

For project 2, in order to be time efficient, you can use software like Qiskit, Pennylane, qBraid and/or other

Overarching motivation

We will start with a reminder on Fourier theory and in particular on discrete Fourier transforms (DFTs). There are many motivations for the DFTs. For those of you familiar with signal processing, harmonic oscillations, and many other areas of applications, Fourier transforms are almost standard kitchen items.
But first some general motivation for Quantum Fourier Transforms (QFTs).

Quantum Fourier Transforms (QFTs)

1. QFTs are the quantum analogue of the Discrete Fourier Transforms (DFTs).
2. They play a crucial role in quantum algorithms like Shor's Algorithm and Quantum Phase Estimation.
3. QFTs provide an *exponential speedup* over classical Fourier Transform methods.

Why Quantum Fourier Transforms and exponential speedup

1. Classical Discrete Fourier Transforms (DFTs) require $O(N^2)$ operations.
2. The Fast Fourier Transform (FFT) improves this to $O(N \log N)$ operations.
3. QFTs reduce complexity to $O((\log N)^2)$ using quantum gates.

Quantum Fourier Transforms and quantum parallelism

1. QFTs act on a *superposition* of states, processing all inputs simultaneously.
2. This is crucial in algorithms like:
   - 2.1 Shor's Algorithm for factoring large numbers.
   - 2.2 Quantum Phase Estimation (QPE) for eigenvalue extraction.

Quantum Fourier Transforms and implementations

1. QFTs require only *Hadamard gates and controlled-phase gates*.
2. A 3-qubit QFT circuit uses only $O(n^2)$.
3. This makes QFTs highly efficient for *quantum hardware*.

Quantum Fourier Transform and Applications in Quantum Computing

1. **Shor's Algorithm:** Uses QFTs to find periodicity in modular exponentiation.
2. **Quantum Phase Estimation (QPE):** QFTs extract eigenvalues of unitary matrices.
3. **Quantum Signal Processing:** QFTs enable spectral analysis on quantum data. Important for AI applications.

Quantum Fourier Transforms and Reduced Memory Complexity (not covered)

1. Classical DFTs require $O(N)$ space.
2. QFT store Fourier-transformed coefficients *implicitly* in qubit amplitudes.
3. Only $O(n)$ qubits are needed for a size $N = 2^n$ transformation.

Quantum Fourier Transforms and Quantum Signal Processing (not covered in this course)

QFTs enable applications such as:

1. Quantum spectral analysis.
2. Quantum image processing.
3. Quantum filtering and denoising.

These can enhance AI, cryptography, and data processing.

Why Quantum Fourier Transforms? Brief summary

1. Quantum Fourier Transforms provide *exponential speedup*
2. They enable key quantum algorithms like *Shor's Algorithm* and *Quantum Phase Estimation*.
3. QFTs are more efficient in *memory usage and circuit complexity* than classical methods.
4. Future quantum applications in *signal processing, AI, and cryptography* will heavily rely on QFT.

Now technicalities: Reminder on Fourier theory, a familiar case first

For problems with so-called harmonic oscillations, given by for example the following differential equation

$$m\frac{d^2x}{dt^2} + \eta\frac{dx}{dt} + x(t) = f(t),$$

where $f(t)$ is an applied external force acting on the system (often called a driving force), one can use the theory of Fourier transformations to find the solutions of this type of equations.

Several driving forces

If one has several driving forces, $f(t) = \sum_n f_n(t)$, one can find the particular solution $x_{pn}(t)$ to the above differential equation for each f_n. The particular solution for the entire driving force is then given by a series like

$$x_p(t) = \sum_n x_{pn}(t).$$

This is known as the principle of superposition. It only applies when the homogenous equation is linear. Superposition is especially useful when $f(t)$ can be written as a sum of sinusoidal terms, because the solutions for each sinusoidal (sine or cosine) term is analytic.

Periodicity

Driving forces are often periodic, even when they are not sinusoidal. Periodicity implies that for some time t our function repeats itself periodically after a period T, that is

$$f(t + \tau) = f(t).$$

One example of a non-sinusoidal periodic force is a square wave. Many components in electric circuits are non-linear, for example diodes. This makes many wave forms non-sinusoidal even when the circuits are being driven by purely sinusoidal sources.

Simple Code Example

The code here shows a typical example of such a square wave generated using the functionality included in the **scipy** Python package. We have used a period of $\tau = 0.2$.

```
import numpy as np
import math
from scipy import signal
import matplotlib.pyplot as plt

# number of points
n = 500
# start and final times
t0 = 0.0
tn = 1.0
# Period
t = np.linspace(t0, tn, n, endpoint=False)
SqrSignal = np.zeros(n)
SqrSignal = 1.0+signal.square(2*np.pi*5*t)
plt.plot(t, SqrSignal)
plt.ylim(-0.5, 2.5)
plt.show()
```

Continuous Fourier transforms and the principle of Superposition

It was Fourier's idea to expand a continuous and periodic function $f(t)$ in terms of sums sine and cosine ordered functions (we will use exponentials however) as

$$f(t) = \sum_{k=1}^{n} a_k \sin(2\pi kt + \phi_n),$$

with ϕ_n being a constant phase. The function f is assumed to be bounded in order to be able to define properly the error in truncating the sum over n, that is

$$\int_a^b dt|f(t)|^2 \leq M < \infty.$$

Below we discuss how to find the coefficients a_n.

Rewriting in terms of sines and cosines

It is common to rewrite the above sum in terms of sines and cosines and to add a complex constant a_0. This gives us

$$f(t) = \frac{a_0}{2} + \sum_{k=1}^{n} \left(a_k \cos\left(2\pi k t\right) + b_n \sin\left(2\pi k t\right) \right).$$

Using the standard trigonometric relations

$$\cos t = \frac{\exp \imath t + \exp -\imath t}{2} \qquad \sin t = \frac{\exp \imath t - \exp -\imath t}{2\imath}.$$

Exponential expression

We can rewrite the Fourier expansion as

$$f(t) = \sum_{k=-n}^{n} c_k \exp(2\pi\imath kt),$$

with $c_0 = a_0/2$. The coefficients c_k are complex and satisfy $c_{-k} = c_k^*$. The constant $c_0 = c_0^*$ is a real number.
The above sum can also be rewritten in terms of the real part of the exponentials as

$$f(t) = 2\mathrm{Re}\left(\sum_{k=0}^{n} c_k \exp(2\pi\imath kt\right),$$

where we used that $c_{-k} = c_k^*$. We leave it as an exercise to the reader to show the latter expression.
How do we determine the coefficients c_k?

Determining c_k

Let us assume that we have a periodic function $f(t)$

$$f(t) = \sum_{k=-n}^{n} c_k \exp(2\pi\imath kt),$$

and we select the coefficient c_l, and multiply the l.h.s and r.h.s. with $\exp(-2\pi\imath lt)$, and isolate the c_l terms

$$c_l = f(t)\exp(-2\pi\imath lt) - \sum_{k=-n, k\neq l}^{n} c_k \exp(2\pi\imath(k-l)t).$$

Integrating both sides

Then we integrate both sides (note we have assumed a period of one) from 0 to 1. This gives

$$c_l = \int_0^1 dt f(t) \exp(-2\pi\imath l t),$$

since the terms with the sum is zero. We leave again this derivation as an exercise to the dedicated reader. If the function f is real (t nd dt are real), then $c_l^* = c_{-l}$. The coefficients c_n are normally called the Fourier coefficients and it is common to relabel them in terms of the function f as

$$\hat{f}(k) = \int_0^1 dt f(t) \exp(-2\pi\imath k t).$$

Independence of interval length

In the discussions and equations above, we have assumed that the integration interval has a length of one, that is a period of length one. It is easy to change this length and as we show below, if we integrate from say a to $a+1$, this is the same as integrating over an interval of length 1.
To show this, we need also to recall that our function $f(t)$ is assumed to be periodic, that is we need to satisfy

$$f(a+1) = f(a).$$

Later we will generalize this to a period of arbitrary length.
Assume that a is any number and taking the derivative of the integral with respect to a, we have

$$\frac{d}{da}\left[\int_a^{a+1} dt f(t)\exp\left(-2\pi\imath kt\right)\right].$$

Writing out the various terms

From the last equation we have then

$$\exp(-2\pi\imath ka)\exp(-2\pi\imath k)f(a+1) - \exp(-2\pi\imath ka)f(a) = 0,$$

where we used the periodicity $f(a+1) = f(a)$ and that $\exp(-2\pi\imath k) = 1$ since k is an integer. This shows that the expression for $\hat{f}$ is independent of a. A common instance is

$$\hat{f}(k) = \int_{-\frac{1}{2}}^{\frac{1}{2}} dt f(t)\exp(-2\pi\imath kt).$$

Changing period

What if the period is not equal to one? Assume we are working with a function $f(t)$ whose period is T. We can then define a function $g(t)$ with period one as

$$g(t) = f(Tt),$$

that is

$$g(t) = \sum_{k=-n}^{n} c_n \exp(2\pi\imath kt),$$

and introducing the variable $s = Tt$ we have $g(t) = f(s)$ we have

$$f(s) = g(t) = \sum_{k=-n}^{n} c_n \exp(2\pi\imath kt) = \sum_{k=-n}^{n} c_k \exp(2\pi\imath ks/T).$$

Harmonics

The so-called harmonics are now defined as $\exp(2\pi\imath ks/T)$. We have

$$\hat{g}(k) = \int_0^1 dtg(t)\exp(-2\pi\imath kt),$$

and using $s = Tt$ we have

$$\hat{g}(k) = \int_0^1 dtg(t)\exp(-2\pi\imath kt) = \frac{1}{T}\int_0^T dsf(s)\exp(-2\pi\imath ks/T).$$

For a period T we have thus the general expression for the Fourier transform

$$\hat{f}(k) = c_k = \frac{1}{T}\int_0^T dtf(t)\exp(-2\pi\imath kt/T).$$

Typical interval

An often used choice of interval is

$$\hat{f}(k) = c_k = \frac{1}{T}\int_{-T/2}^{T/2} dt f(t) \exp\left(-2\pi\imath kt/T\right),$$

and a common choice for the harmonics is $(1/\sqrt{T})\exp\left(2\pi\imath kt/T\right)$. As a small addendum, if the signal (function) $f(t)$ is real and even one has $f(-t) = f(t)$. If f is an even function, then $\hat{f}$ is also an even function which leads to $\hat{f}(-k) = \hat{f}(k)$.

Sinusoidal example

For the sinusoidal example the period is $T = 2\pi/\omega$. However, higher harmonics can also satisfy the periodicity requirement. In general, any force that satisfies the periodicity requirement can be expressed as a sum over harmonics,

$$f(t) = \frac{a_0}{2} + \sum_{n>0} a_n \cos(2n\pi t/T) + b_n \sin(2n\pi t/T).$$

Square well example

In the code discussed earlier, we used a square well as a our example. If we assume our period has length 1, we can define the square well function $f(t)$ as

$$f(t) = \begin{cases} 1 & 0 \leq t \leq \frac{1}{2} \\ -1 & \frac{1}{2} < t \leq 1 \end{cases}$$

The zeroth coefficient a_0 is zero since it is the average of the function $f(t)$ over the intergration domain $t \in [0, 1]$.

The coefficients

We find that the other coefficients are

$$\hat{f}(n) = c_n = \int_0^1 dt f(t) \exp(-2\pi\imath nt) = \frac{1}{\imath\pi n}(1 - \exp -\imath\pi n).$$

We note that $(1 - \exp -\imath\pi n)$ is zero when n is an even number and 2 when n is an odd number. We can then combine the positive and negative values of n using

$$\exp(2\pi\imath nt) - \exp(-2\pi\imath nt) = 2\imath \sin(2\pi nt),$$

and we have

$$f(t) = \frac{4}{\pi}\sum_{k=0}^{\infty}\frac{1}{2k+1}\sin(2\pi(2k+1)t).$$

Code for the Fourier Transforms

The code here uses the Fourier series applied to a square wave signal. The code here visualizes the various approximations given by Fourier series compared with a square wave with period $T = 0.2$ (dimensionless time), width 0.1 and max value of the force $F = 2$. We see that when we increase the number of components in the Fourier series, the Fourier series approximation gets closer and closer to the square wave signal.

```
import numpy as np
import math
from scipy import signal
import matplotlib.pyplot as plt

# number of points
n = 500
# start and final times
t0 = 0.0
tn = 1.0
# Period
T =0.2
# Max value of square signal
Fmax= 2.0
# Width of signal
Width = 0.1
t = np.linspace(t0, tn, n, endpoint=False)
SqrSignal = np.zeros(n)
```

Inverse Fourier transform

The inverse Fourier transform is given by

$$\boldsymbol{F}^{-1}[g(y)] = \frac{1}{2\pi}\int_{-\infty}^{\infty} d\omega \exp i\omega y g(\omega).$$

The inverse Fourier transform of the product of the two functions $\hat{f}\hat{g}$ can be written as

$$\boldsymbol{F}^{-1}[(\hat{f}\hat{g})(x)] = \frac{1}{2\pi}\int_{-\infty}^{\infty} d\omega \exp i\omega x \hat{f}(\omega)\hat{g}(\omega).$$

Rewriting

We can rewrite the latter as

$$\boldsymbol{F}^{-1}[(\hat{f}\hat{g})(x)] = \int_{-\infty}^{\infty} d\omega \exp i\omega x \hat{f}(\omega) \left[\frac{1}{2\pi}\int_{-\infty}^{\infty} g(y)dy \exp -i\omega y\right] = \frac{1}{2\pi}$$

which is simply

$$\boldsymbol{F}^{-1}[(\hat{f}\hat{g})(x)] = \int_{-\infty}^{\infty} dy g(y) f(x-y) = (f * g)(x),$$

the convolution of the functions f and g.

Transforming to discrete variables

In the Fourier transform c_n is transformed from a dicrete variable to a continuous one as $T \to \infty$. We then replace c_n with $f(k)dk$ and let $n/T \to k$, and the sum is changed to an integral. This gives

$$f(x) = \int_{-\infty}^{\infty} dk F(k) \exp i(2\pi kx)$$

and

$$F(k) = \int_{-\infty}^{\infty} dx f(x) \exp -i(2\pi kx)$$

One way to interpret the Fourier transform is then as a transformation from one basis to another.

Discrete Fourier transform

We change notation by replacing t with x.
Next we make another generalization by having a discrete function, that is $f(x) \to f(x_k)$ with $x_k = k\Delta x$ for $k = 0, \ldots, N-1$. This leads to the sums

$$f_x = \frac{1}{n} \sum_{k=0}^{n-1} F_k \exp i(2\pi kx)/n,$$

and

$$F_k = \sum_{x=0}^{n-1} f_x \exp -i(2\pi kx)/n.$$

Although we have used functions here, this could also be a set of numbers.

Simple example

As an example we can have a set of complex numbers $\{x_0, \ldots, x_{n-1}\}$ with fixed length n, we can Fourier transform this as

$$y_k = \frac{1}{\sqrt{n}} \sum_{j=0}^{n-1} x_j \exp i(2\pi jk)/n,$$

leading to a new set of complex numbers $\{y_0, \ldots, y_{n-1}\}$.

Discrete Fourier Transformations

Consider two sets of complex numbers x_k and y_k with $k = 0, 1, \ldots, n-1$ entries. The discrete Fourier transform is defined as

$$y_k = \frac{1}{\sqrt{n}} \sum_{j=0}^{n-1} \exp\left(\frac{2\pi \imath jk}{n}\right) x_j.$$

As an example, assume $x_0 = 1$ and $x_1 = 1$. We can then use the above expression to find y_0 and y_1.
With the above formula we get then

$$\begin{aligned} y_0 &= \frac{1}{\sqrt{2}} \left(\exp\left(\frac{2\pi \imath 0 \times 1}{2}\right) \times 1 + \exp\left(\frac{2\pi \imath 0 \times 1}{2}\right) \times 2 \right) \\ &= \frac{1}{\sqrt{2}}(1+2) = \frac{3}{\sqrt{2}}, \end{aligned}$$

and

$$\begin{aligned} y_1 &= \frac{1}{\sqrt{2}} \left(\exp\left(\frac{2\pi \imath 0 \times 1}{2}\right) \times 1 + \exp\left(\frac{2\pi \imath 1 \times 1}{2}\right) \times 2 \right) = \\ &= \frac{1}{\sqrt{2}}(1 + 2\exp(\pi \imath)) = -\frac{1}{\sqrt{2}}. \end{aligned}$$

More details on Discrete Fourier transforms

Suppose that we have a vector f of n complex numbers, $f_k, k \in \{0, 1, \ldots, n-1\}$. Then the discrete Fourier transform (DFT) is a map from these n complex numbers to n complex numbers, the Fourier transformed coefficients $\tilde{f}_j$, given by

$$\tilde{f}_j = \frac{1}{\sqrt{n}} \sum_{k=0}^{n-1} \omega^{-jk} f_k$$

where $\omega = \exp\left(\frac{2\pi i}{N}\right)$.

Inverse DFT

The inverse DFT is given by

$$f_j = \frac{1}{\sqrt{n}} \sum_{k=0}^{n-1} \omega^{jk} \tilde{f}_k$$

To see this consider how the basis vectors transform. If $f_k^l = \delta_{k,l}$, then

$$\tilde{f}_j^l = \frac{1}{\sqrt{n}} \sum_{k=0}^{n-1} \omega^{-jk} \delta_{k,l} = \frac{1}{\sqrt{n}} \omega^{-jl}$$

Orthonormality

These DFT vectors are orthonormal, that is

$$\sum_{j=0}^{n-1} \tilde{f}^{l*}_j \tilde{f}^m_j = \frac{1}{n} \sum_{j=0}^{n-1} \omega^{jl} \omega^{-jm} = \frac{1}{N} \sum_{j=0}^{n-1} \omega^{j(l-m)}$$

This last sum can be evaluated as a geometric series, but beware of the $(l - m) = 0$ term, and yields

$$\sum_{j=0}^{n-1} \tilde{f}^{l*}_j \tilde{f}^m_j = \delta_{l,m}$$

Inverse transform

From this we can check that the inverse DFT does indeed perform the inverse transform:

$$f_j = \frac{1}{\sqrt{n}} \sum_{k=0}^{n-1} \omega^{jk} \tilde{f}_k = \frac{1}{\sqrt{n}} \sum_{k=0}^{n-1} \omega^{jk} \frac{1}{\sqrt{n}} \sum_{l=0}^{n-1} \omega^{-lk} f_l = \frac{1}{n} \sum_{k,l=0}^{n-1} \omega^{(j-l)k} f_l = \sum_{l=0}^{n-1} \delta$$

Summary: Discrete Fourier transforms

The Discrete Fourier Transform is a tool used in many different fields and finds applications in for example signal processing. It has many many applications (in Speech Recognition, Data Processing, Optics, Acoustics). It is used in any application which performs Digital Signal Processing.
There is an efficient algorithm, called the Fast Fourier Transform (FFT), which uses the special relationship amongst the roots of unity to compute the entire DFT in $O(n \log n)$ time. Moreover, it is possible to take the DFT and recover the original polynomial in its coefficient form in $O(n \log n)$ time, by using the FFT to compute the inverse DFT.

Fast Fourier transform (FFT)

Because of the many important applications of DFT, the Fast Fourier Transform is ranked among the top ten algorithms in the 20th century. It came to light as a tool for Signal processing in 1965, when Cooley (of IBM) and Tukey (of Princeton) wrote a paper presenting the algorithm. However the basics of the algorithm were known long before that, for example, it was known to Gauss back in 1805. To read more about Fast Fourier transforms and similar topics, see for example Fast Fourier Transform - Algorithms and Applications. See also `https://github.com/CompPhysics/QuantumComputingMachineLearning/blob/gh-pages/doc/Textbooks/fastfourier.pdf`
Our emphasis is on the link between discrete Fourier transforms and quantum Fourier transforms. For fast Fourier transforms, we may (if there is interest) have a dedicated sessions on).

More summary: The discrete Fourier transform (DFT)

The discrete Fourier transform takes as input a complex vector

$$\boldsymbol{x} = |\boldsymbol{x}\rangle = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_{n-2} \\ x_{n-1} \end{bmatrix}.$$

Each entry of the output vector $|\boldsymbol{y}\rangle$ is given by

$$y_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x_j e^{2\pi i j k / N}.$$

Output vector

The output is a complex vector

$$|\boldsymbol{y}\rangle = \frac{1}{\sqrt{n}} \begin{bmatrix} \sum_{j=0}^{n-1} x_j e^{2\pi i j 0/n} \\ \sum_{j=0}^{n-1} x_j e^{2\pi i j 1/n} \\ \sum_{j=0}^{n-1} x_j e^{2\pi i j 2/n} \\ \vdots \\ \sum_{j=0}^{n-1} x_j e^{2\pi i j (n-2)/n} \\ \sum_{j=0}^{n-1} x_j e^{2\pi i j (n-1)/n} \end{bmatrix}$$

From DFT to QFT

The Quantum Fourier Transform (QFT) has mathematically the same equation as starting point but the notation is generally different. We generally compute the quantum Fourier transform on a set of orthonormal basis state vectors $|0\rangle, |1\rangle, ..., |N-1\rangle$. The linear operator defining the transform is given by the action on basis states

$$|j\rangle \mapsto \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi ijk/N} |k\rangle.$$

In terms of arbitrary states

This can be written on arbitrary states,

$$\sum_{j=0}^{N-1} x_j |j\rangle \mapsto \sum_{k=0}^{N-1} y_k |k\rangle$$

where each amplitude y_k is the discrete Fourier transform of x_j.

Unitarity

The quantum Fourier transform is unitary. Taking $N = 2^n$, for n qubits gives us the orthonormal (computational) basis

$$|0\rangle, |1\rangle, ..., |2^n - 1\rangle.$$

Binary representation

Each of the computational basis states can be represented in binary

$$j = j_1 j_2 \cdots j_n$$

where each j_k is either 0 or 1, and the corresponding binary vector is $|j_1 j_2 \cdots j_n\rangle$.

Rewriting the QFT

The quantum Fourier transform on one of these n-qubit vectors can be written as,

$$|j_1 j_2 \cdots j_n\rangle = \frac{\left(|0\rangle + e^{2\pi i 0.j_n}|1\rangle\right) \otimes \left(|0\rangle + e^{2\pi i 0.j_{n-1}j_n}|1\rangle\right) \otimes \cdots \otimes \left(|0\rangle + e^{2\pi}\right.}{2^{n/2}}$$

Short hand notation

In the above, we use the notation

$$0.j_l j_{l+1} \cdots j_n = \frac{j+l}{2} + \frac{j_{l+1}}{2^2} + \cdots + \frac{j_n}{2^{m-l+1}}$$

Two-qubit system

First a two qubit system. Notes to be added.

Four qubit system

Let us then move to a four qubit system. The basis states are,

$$|j_1 j_2 j_3 j_4\rangle$$

where j_k is either 0 or 1. We have

$$\begin{aligned}
0.j_3 &= \frac{j_3}{2} \\
0.j_2 j_3 &= \frac{j_2}{2} + \frac{j_3}{4} \\
0.j_1 j_2 j_3 &= \frac{j_1}{2} + \frac{j_2}{4} + \frac{j_3}{8} \\
0.j_0 j_1 j_2 j_3 &= \frac{j_0}{2} + \frac{j_1}{4} + \frac{j_2}{8} + \frac{j_3}{16}
\end{aligned}$$

QFT acts as follows

The quantum Fourier transform acts as follows:

$$|j_1 j_2 j_3 j_4\rangle \mapsto \frac{1}{\sqrt{2^{4/2}}} \left(|0\rangle + e^{2\pi i 0.j4}|1\rangle\right) \otimes \left(|0\rangle + e^{2\pi i 0.j_3 j4}|1\rangle\right) \otimes \left(|0\rangle + e^{2\pi i 0}\right.$$

Quantum circuits

To compose a quantum circuit that calculates the quantum Fourier transform we use the operators

$$R_k = \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{bmatrix}.$$

For the circuit computing the QFT on four qubits, see whiteboard notes again.

Rotation gates

In this example, the R_k gates are:

$$R_1 = \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^0} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad R_2 = \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^2} \end{bmatrix}, \quad R_3 = \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^3} \end{bmatrix}$$

Quantum Fourier transform, more material

We have defined the QFT in terms of the unitary operation

$$|\psi'\rangle \leftarrow \hat{F}|\psi\rangle, \quad \hat{F}^\dagger \hat{F} = I$$

Orthonormal basis

In terms of an orthonormal basis $|0\rangle, |1\rangle, \ldots, |0\rangle$ this linear operator has the following action

$$|j\rangle \rightarrow \sum_{k=0}^{N-1} \exp i(2\pi jk/N)|k$$

or on an arbitrary state

$$\sum_{j=0}^{N-1} x_j|j\rangle \rightarrow \sum_{k=0}^{N-1} y_k|k\rangle$$

equivalent to the equation for discrete Fourier transform on a set of complex numbers.

Using computational basis

Next we assume an n-qubit system, where we take $N = s^n$ in the computational basis

$$|0\rangle, \ldots, |2^n - 1\rangle.$$

We make use of the binary representation $j = j_1 2^{n-1} + j_2 2^{n-2} + \cdots + j_n 2^0$, and take note of the notation $0.j_l j_{l+1} \ldots j_m$ representing the binary fraction $\frac{j_l}{2^1} + \frac{j_{l+1}}{2^2} + \cdots + \frac{j_m}{2^{m-l+1}}$. With this we define the product representation of the quantum Fourier transform

$$|j_1, \ldots, j_n\rangle \rightarrow \frac{(|0\rangle + \exp i(2\pi 0.j_n))\,(|0\rangle + \exp i(2\pi 0.j_{j-1} j_n)) \ldots (|0\rangle + \exp$$
$$2^{n/2}$$

Components

From the product representation we can derive a circuit for the quantum Fourier transform. This will make use of the following two single-qubit gates

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

$$R_k = \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{bmatrix}$$

Using the Hadamard gate

The Hadamard gate on a single qubit creates an equal superposition of its basis states, assuming it is not already in a superposition, such that

$$H|0\rangle = \frac{1}{\sqrt{2}}\left(|0\rangle + |1\rangle\right), \quad H|1\rangle = \frac{1}{\sqrt{2}}\left(|0\rangle - |1\rangle\right)$$

The R_k gate simply adds a phase if the qubit it acts on is in the state $|1\rangle$

$$R_k|0\rangle = |0\rangle, \quad R_k|1\rangle = e^{2\pi i/2^k}|1\rangle$$

Since all this gates are unitary, the quantum Fourier transfrom is also unitary.

Algorithm

Assume we have a quantum register of n qubits in the state $|j_1 j_2 \dots j_n\rangle$. Applying the Hadamard gate to the first qubit produces the state

$$H|j_1 j_2 \dots j_n\rangle = \frac{\left(|0\rangle + e^{2\pi i 0.j_1}|1\rangle\right)}{2^{1/2}}|j_2 \dots j_n\rangle.$$

Binary fraction

Here we have made use of the binary fraction to represent the action of the Hadamard gate

$$\exp 2\pi i 0.j_1 = -1,$$

if $j_1 = 1$ and $+1$ if $j_1 = 0$.

Controlled rotation gate

Furthermore we can apply the controlled-R_k gate, with all the other qubits j_k for $k > 1$ as control qubits to produce the state

$$\frac{\left(|0\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n}|1\rangle\right)}{2^{1/2}}|j_2 \ldots j_n\rangle$$

Next we do the same procedure on qubit 2 producing the state

$$\frac{\left(|0\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n}|1\rangle\right)\left(|0\rangle + e^{2\pi i 0.j_2 \ldots j_n}|1\rangle\right)}{2^{2/2}}|j_2 \ldots j_n\rangle$$

Applying to all qubits

Doing this for all n qubits yields state

$$\frac{\left(|0\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n}|1\rangle\right)\left(|0\rangle + e^{2\pi i 0.j_2 \ldots j_n}|1\rangle\right)\ldots\left(|0\rangle + e^{2\pi i 0.j_n}|1\rangle\right)}{2^{n/2}}|j_2 \ldots j_n\rangle$$

At the end we use swap gates to reverse the order of the qubits

$$\frac{\left(|0\rangle + e^{2\pi i 0.j_n}|1\rangle\right)\left(|0\rangle + e^{2\pi i 0.j_{n-1} j_n}|1\rangle\right)\ldots\left(|0\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n}|1\rangle\right)}{2^{n/2}}|j_2 \ldots j_n\rangle$$

This is just the product representation from earlier, obviously our desired output.

Quantum Fourier transform

Now lets turn to the Quantum Fourier transform (QFT). We've already seen the QFT for $N = 2$. It is the Hadamard transform:

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

QFT for $N = 2$

Why is this the QFT for $N = 2$? Well suppose have the single qubit state $a_0|0\rangle + a_1|1\rangle$. If we apply the Hadamard operation to this state we obtain the new state

$$\frac{1}{\sqrt{2}}(a_0 + a_1)|0\rangle + \frac{1}{\sqrt{2}}(a_0 - a_1)|1\rangle = \tilde{a}_0|0\rangle + \tilde{a}_1|1\rangle.$$

In other words the Hadamard gate performs the DFT for $N = 2$ on the amplitudes of the state! Notice that this is very different that computing the DFT for $N = 2$: remember the amplitudes are not numbers which are accessible to us mere mortals, they just represent our description of the quantum system.

Full QFT

So what is the full quantum Fourier transform? It is the transform which takes the amplitudes of a N dimensional state and computes the Fourier transform on these amplitudes (which are then the new amplitudes in the computational basis.) In other words, the QFT enacts the transform

$$\sum_{x=0}^{N-1} a_x|x\rangle \rightarrow \sum_{x=0}^{N-1} \tilde{a}_x|x\rangle = \sum_{x=0}^{N-1} \frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} \omega_N^{-xy} a_y|x\rangle$$

Explicit transform

It is easy to see that this implies that the QFT performs the following transform on basis states:

$$|x\rangle \rightarrow \frac{1}{\sqrt{N}} \sum_{y=0}^{N-1} \omega_N^{-xy} |y\rangle$$

Thus the QFT is given by the matrix

$$U_{QFT} = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} \omega_N^{-yx} |y\rangle\langle x|$$

Unitarity

The last matrix is unitary. Let's check this:

$$
\begin{aligned}
U_{QFT}\,U_{QFT}^{\dagger} &= \frac{1}{N}\sum_{x=0}^{N-1}\sum_{y=0}^{N-1}\omega_N^{yx}|x\rangle\left\langle y\right|\sum_{x'=0}^{N-1}\sum_{y'=0}^{N-1}\omega_N^{-y'x'}\left|y'\right\rangle\langle x'| \\
&= \frac{1}{N}\sum_{x,y,x',y'=0}^{N-1}\omega_N^{yx-y'x'}\delta_{y,y'}|x\rangle\left\langle x'\right| = \frac{1}{N}\sum_{x,y,x'=0}^{N-1}\omega_N^{y(x-x')}\left|x\right\rangle \\
&= \sum_{x,x'=0}^{N-1}\delta_{x,x'}|x\rangle\langle x'| = I
\end{aligned}
$$

Importance of QFT

The QFT is a very important transform in quantum computing. It can be used for all sorts of cool tasks, including, as we shall see in Shor's algorithm. But before we can use it for quantum computing tasks, we should try to see if we can efficiently implement the QFT with a quantum circuit. Indeed we can and the reason we can is intimately related to the fast Fourier transform.

Circuit QFT

Let's derive a circuit for the QFT when $N = 2^n$. The QFT performs the transform

$$|x\rangle \rightarrow \frac{1}{\sqrt{2^n}} \sum_{y=0}^{2^n-1} \omega_N^{-xy} |y\rangle$$

Then we can expand out this sum

$$|x\rangle \rightarrow \frac{1}{\sqrt{2^n}} \sum_{y_1,y_2,\ldots,y_n \in \{0,1\}} \omega_N^{-x \sum_{k=1}^{n} 2^{n-k} y_k} |y_1, y_2, \ldots, y_n\rangle$$

Expanding the exponential

Expanding the exponential of a sum to a product of exponentials and collecting these terms in from the appropriate terms we can express this as

$$|x\rangle \rightarrow \frac{1}{\sqrt{2^n}} \sum_{y_1,y_2,\ldots,y_n \in \{0,1\}} \bigotimes_{k=1}^{n} \omega_N^{-x2^{n-k}y_k} |y_k\rangle$$

Rearranging

We can rearrange the sum and products as

$$|x\rangle \rightarrow \frac{1}{\sqrt{2^n}} \bigotimes_{k=1}^{n} \left[\sum_{y_k \in \{0,1\}} \omega_N^{-x 2^{n-k} y_k} |y_k\rangle \right]$$

Expanding this sum yields

$$|x\rangle \rightarrow \frac{1}{\sqrt{2^n}} \bigotimes_{k=1}^{n} \left[|0\rangle + \omega_N^{-x 2^{n-k}} |1\rangle \right]$$

Notation for binary fraction

But now notice that $\omega_N^{-x2^{n-k}}$ is not dependent on the higer order bits of x. It is convenient to adopt the following expression for a binary fraction:

$$0.x_l x_{l+1} \ldots x_n = \frac{x_l}{2} + \frac{x_{l+1}}{4} + \cdots + \frac{x_n}{2^{n-l+1}}$$

More notations

Then we can see that

$$|x\rangle \rightarrow \frac{1}{\sqrt{2^n}} \left[|0\rangle + e^{-2\pi i 0.x_n}|1\rangle\right] \otimes \left[|0\rangle + e^{-2\pi i 0.x_{n-1}x_n}|1\rangle\right] \otimes \cdots \otimes \left[|0\rangle + e^{-2\pi}\right.$$

This is a very useful form of the QFT for $N = 2^n$. Why? Because we see that only the last qubit depends on the the values of all the other input qubits and each further bit depends less and less on the input qubits. Further we note that $e^{-2\pi i 0.a}$ is either +1 or -1 , which reminds us of the Hadamard transform.

Deriving a circuit

So how do we use this to derive a circuit for the QFT over $N = 2^n$? Take the first qubit of $|x_1, \ldots, x_n\rangle$ and apply a Hadamard transform. This produces the transform

$$|x\rangle \to \frac{1}{\sqrt{2}} \left[|0\rangle + e^{-2\pi i 0 \cdot x_1}|1\rangle\right] \otimes |x_2, x_3, \ldots, x_n\rangle$$

Rotation gate

Now define the rotation gate

$$R_k = \begin{bmatrix} 1 & 0 \\ 0 & \exp\left(\frac{-2\pi i}{2^k}\right) \end{bmatrix} \tag{30}$$

If we now apply controlled R_2, R_3, etc. gates controlled on the appropriate bits this enacts the transform

$$|x\rangle \rightarrow \frac{1}{\sqrt{2}}\left[|0\rangle + e^{-2\pi i 0.x_1 x_2 \ldots x_n}|1\rangle\right] \otimes |x_2, x_3, \ldots, x_n\rangle$$

Proceeding

Thus we have reproduced the last term in the QFTed state. Of course now we can proceed to the second qubit, perform a Hadamard, and the appropriate controlled R_k gates and get the second to last qubit. Thus when we are finished we will have the transform

$$|x\rangle \rightarrow \frac{1}{\sqrt{2^n}} \left[|0\rangle + e^{-2\pi i 0.x_1 x_2 \cdots x_n}|1\rangle\right] \otimes \left[|0\rangle + e^{-2\pi i 0.x_1 x_2 \cdots x_{n-1}}|1\rangle\right] \otimes \cdots \otimes \left[|0\right.$$

Reversing the order of these qubits will then produce the QFT!

QFT

The Quantum Fourier Transform (QFT) has mathematically the same equation as starting point but the notation is generally different. We generally compute the quantum Fourier transform on a set of orthonormal basis state vectors $|0\rangle, |1\rangle, ..., |N-1\rangle$. The linear operator defining the transform is given by the action on basis states

$$|j\rangle \mapsto \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi ijk/N} |k\rangle.$$

In terms of arbitrary states

This can be written on arbitrary states,

$$\sum_{j=0}^{N-1} x_j |j\rangle \mapsto \sum_{k=0}^{N-1} y_k |k\rangle$$

where each amplitude y_k is the discrete Fourier transform of x_j.

Unitarity

The quantum Fourier transform is unitary. Taking $N = 2^n$, for n qubits gives us the orthonormal (computational) basis

$$|0\rangle, |1\rangle, ..., |2^n - 1\rangle.$$

Binary representation

Each of the computational basis states can be represented in binary

$$j = j_1 j_2 \cdots j_n$$

where each j_k is either 0 or 1, and the corresponding binary vector is $|j_1 j_2 \cdots j_n\rangle$.

Rewriting the QFT

The quantum Fourier transform on one of these n-qubit vectors can be written as,

$$|j_1 j_2 \cdots j_n\rangle = \frac{\left(|0\rangle + e^{2\pi i 0.j_n}|1\rangle\right) \otimes \left(|0\rangle + e^{2\pi i 0.j_{n-1}j_n}|1\rangle\right) \otimes \cdots \otimes \left(|0\rangle + e^{2\pi}\right.}{2^{n/2}}$$

Short hand notation

In the above, we use the notation

$$0.j_l j_{l+1} \cdots j_n = \frac{j_l}{2} + \frac{j_{l+1}}{2^2} + \cdots + \frac{j_n}{2^{n-l+1}}$$

If we start counting from zero, that is we have instead

$$|j_0 j_1 \cdots j_{n-1}\rangle = \frac{\left(|0\rangle + e^{2\pi i 0.j_{n-1}}|1\rangle\right) \otimes \left(|0\rangle + e^{2\pi i 0.j_{n-2} j_n}|1\rangle\right) \otimes \cdots \otimes \left(|0\rangle +}{2^{n/2}}$$

we get

$$0.j_l j_{l+1} \cdots j_{n-1} = \frac{j_l}{2} + \frac{j_{l+1}}{2^2} + \cdots + \frac{j_{n-1}}{2^{n-l}}$$

Four qubit system

The basis states are

$$|j_1 j_2 j_3 j_4\rangle$$

where j_k is either 0 or 1. We have

$$\begin{aligned}
0.j_3 &= \frac{j_3}{2} \\
0.j_2 j_3 &= \frac{j_2}{2} + \frac{j_3}{4} \\
0.j_1 j_2 j_3 &= \frac{j_1}{2} + \frac{j_2}{4} + \frac{j_3}{8} \\
0.j_0 j_1 j_2 j_3 &= \frac{j_0}{2} + \frac{j_1}{4} + \frac{j_2}{8} + \frac{j_3}{16}
\end{aligned}$$

QFT acts as follows

The quantum Fourier transform acts as follows:

$$|j_1 j_2 j_3 j_4\rangle \mapsto \frac{1}{\sqrt{2^{4/2}}} \left(|0\rangle + e^{2\pi i 0.j4}|1\rangle\right) \otimes \left(|0\rangle + e^{2\pi i 0.j_3 j4}|1\rangle\right) \otimes \left(|0\rangle + e^{2\pi i 0}\right.$$

Quantum circuits

To compose a quantum circuit that calculates the quantum Fourier transform we use the operators

$$R_k = \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{bmatrix}.$$

Rotation gates

In this example, the R_k gates are:

$$R_1 = \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^0} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad R_2 = \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^2} \end{bmatrix}, \quad R_3 = \begin{bmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^3} \end{bmatrix}$$

Using the Hadamard gate

The Hadamard gate on a single qubit creates an equal superposition of its basis states, assuming it is not already in a superposition, such that

$$H|0\rangle = \frac{1}{\sqrt{2}}\left(|0\rangle + |1\rangle\right), \quad H|1\rangle = \frac{1}{\sqrt{2}}\left(|0\rangle - |1\rangle\right)$$

The R_k gate simply adds a phase if the qubit it acts on is in the state $|1\rangle$

$$R_k|0\rangle = |0\rangle, \quad R_k|1\rangle = e^{2\pi i/2^k}|1\rangle$$

Since all this gates are unitary, the quantum Fourier transfrom is also unitary.

Algorithm

Assume we have a quantum register of n qubits in the state $|j_1 j_2 \dots j_n\rangle$. Applying the Hadamard gate to the first qubit produces the state

$$H|j_1 j_2 \dots j_n\rangle = \frac{\left(|0\rangle + e^{2\pi i 0.j_1}|1\rangle\right)}{2^{1/2}}|j_2 \dots j_n\rangle.$$

Binary fraction

Here we have made use of the binary fraction to represent the action of the Hadamard gate

$$\exp 2\pi i 0.j_1 = -1,$$

if $j_1 = 1$ and $+1$ if $j_1 = 0$.

Controlled rotation gate

Furthermore we can apply the controlled-R_k gate, with all the other qubits j_k for $k > 1$ as control qubits to produce the state

$$\frac{\left(|0\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n}|1\rangle\right)}{2^{1/2}}|j_2 \ldots j_n\rangle$$

Next we do the same procedure on qubit 2 producing the state

$$\frac{\left(|0\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n}|1\rangle\right)\left(|0\rangle + e^{2\pi i 0.j_2 \ldots j_n}|1\rangle\right)}{2^{2/2}}|j_2 \ldots j_n\rangle$$

Applying to all qubits

Doing this for all n qubits yields state

$$\frac{\left(|0\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n}|1\rangle\right)\left(|0\rangle + e^{2\pi i 0.j_2 \ldots j_n}|1\rangle\right)\ldots\left(|0\rangle + e^{2\pi i 0.j_n}|1\rangle\right)}{2^{n/2}}|j_2 \ldots j_n\rangle$$

At the end we use swap gates to reverse the order of the qubits

$$\frac{\left(|0\rangle + e^{2\pi i 0.j_n}|1\rangle\right)\left(|0\rangle + e^{2\pi i 0.j_{n-1} j_n}|1\rangle\right)\ldots\left(|0\rangle + e^{2\pi i 0.j_1 j_2 \ldots j_n}|1\rangle\right)}{2^{n/2}}|j_2 \ldots j_n\rangle$$

This is just the product representation from earlier, obviously our desired output.